

МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»

Лабораторная работа №3
по курсу «Технологии параллельного программирования»

Сортировка чисел на GPU. Свертка, сканирование, гистограмма.

Выполнил: *Серый Никита Олегович*

Группа: *M8O-409B-22*

Преподаватели: А.Ю. Морозов,
Е.Е. Заяц

Москва, 2025

Условие

1. Цель работы: Ознакомление с фундаментальными алгоритмами GPU: свертка (reduce), сканирование (blelloch scan) и гистограмма (histogram). Реализация одной из сортировок на CUDA. Использование разделяемой и других видов памяти. Исследование производительности программы с помощью утилиты nvprof.
2. Вариант 3. Сортировка подсчетом. Диапазон от 0 до 255. Требуется реализовать сортировку подсчетом для чисел типа uchar. Должны быть реализованы:
 - а. Алгоритм гистограммы, с использованием атомарных операций и разделяемой памяти.
 - б. Алгоритм сканирования, с бесконфликтным использованием разделяемой памяти.
 - в. Ограничения: $n \leq 537 * 10^6$

Пример:

Входной файл (stdin), hex	Выходной файл (stdout), hex
0A 00 00 00 01 02 03 01 02 03 01 02 03 04	01 01 01 02 02 02 03 03 03 04

Программное и аппаратное обеспечение

Графический процессор (Google Colab)

Compute capability	7.5
Name	Tesla T4
Total Global Memory	15828320256
Shared memory per block	49152
Registers per block	65536
Warp size	32
Max threads per block	(1024, 1024, 64)
Max block	(2147483647, 65535, 65535)
Total constant memory	6553
Multiprocessors count	40

Процессор AMD Ryzen 5 5500U (2.1 GHz, max 4 GHz)

Technology	7 нм
Cores	6

Threads	12
Cache (Level 2)	64К (на ядро)
Cache (Level 2)	512К (на ядро)
Cache (Level 3)	8 МБ (всего)

Оперативная память

Number of modules	2
Module Size	8 ГБ
Frequency	3200 МГц
Generation	DDR4

Жесткий диск - 512 ГБ SSD

OS - Ubuntu 22

IDE - VS Code

Compiler - nvcc

Метод решения

Программа реализует параллельную сортировку подсчетом, разделенную на три фазы синхронизации через запуск отдельных CUDA-ядер для обработки данных объемом до $537 \cdot 10^6$ элементов.

Сначала вычисляется гистограмма с применением атомарных операций в разделяемой памяти, затем выполняется сканирование для определения смещений, и в финале происходит параллельное заполнение выходного массива.

Ключевой особенностью является использование иерархии памяти GPU для минимизации задержек и обеспечения бесконфликтного доступа к банкам памяти при выполнении префиксной суммы.

Описание программы

Структура программного комплекса

Для обеспечения модульности и читаемости, проект логически разделен на три уровня:

1. **main**: управление ЖЦ программы, чтение и запись бинарных потоков.
2. **counting_sort**: выделение ресурсов на GPU, пересылка данных и координация запусков ядер.
3. **kernels**: реализация алгоритмов гистограммы, сканирования и расстановки.

Основные типы данных

- `uchar`: размер в 1 байт определяет диапазон значений [0,255] и размер гистограммы.
- `int`: для счетчиков гистограммы и индексов префиксных сумм.
- `cudaEvent_t`: служебный тип CUDA для замера времени выполнения операций на GPU.

Основные функции

- `main()`: точка входа, чтение заголовка и данных из `stdin`, вызов основной функции сортировки.
- `counting_sort(n, h_input, h_output)`: представляет логику работы с API CUDA.
- `CSC(call)`: для перехвата и диагностики ошибок исполнения в реальном времени.

Ядра

```
__global__ void histogram_kernel(const unsigned char* input, int* histogram, int n) {
    __shared__ int s_hist[k_alphabet_size];
    if (threadIdx.x < k_alphabet_size) s_hist[threadIdx.x] = 0;
    __syncthreads();
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    int stride = gridDim.x * blockDim.x;
    for (int i = idx; i < n; i += stride) {
        atomicAdd(&s_hist[input[i]], 1);
    }
    __syncthreads();
    if (threadIdx.x < k_alphabet_size) {
        atomicAdd(&histogram[threadIdx.x], s_hist[threadIdx.x]);
    }
}
```

1. Гистограмма - Использует `__shared__` память для хранения локальных счетчиков блока. Атомарные операции `atomicAdd` обеспечивают корректность при одновременном обращении потоков к одним ячейкам. Это минимизирует нагрузку на глобальную шину памяти.

```
__global__ void scan_kernel(const int* histogram, int* scan) {
    __shared__ int temp[k_alphabet_size];
    temp[threadIdx.x] = histogram[threadIdx.x];
    __syncthreads();
```

```

int offset = 1;
for (int d = k_alphabet_size >> 1; d > 0; d >>= 1) {
    __syncthreads();
    if (threadIdx.x < d) {
        int ai = offset * (2 * threadIdx.x + 1) - 1;
        int bi = offset * (2 * threadIdx.x + 2) - 1;
        temp[bi] += temp[ai];
    }
    offset *= 2;
}
if (threadIdx.x == 0) temp[k_alphabet_size - 1] = 0;
for (int d = 1; d < k_alphabet_size; d *= 2) {
    offset >>= 1;
    __syncthreads();
    if (threadIdx.x < d) {
        int ai = offset * (2 * threadIdx.x + 1) - 1;
        int bi = offset * (2 * threadIdx.x + 2) - 1;
        int t = temp[ai];
        temp[ai] = temp[bi];
        temp[bi] += t;
    }
}
__syncthreads();
scan[threadIdx.x] = temp[threadIdx.x];
}

```

2. Параллельное сканирование - Реализует алгоритм Blelloch Scan. Работает в разделяемой памяти одного блока для обеспечения максимально быстрой синхронизации. Алгоритм спроектирован так, чтобы минимизировать конфликты банков памяти.

```

__global__ void fill_output_kernel(unsigned char* output, const int* scan, int n) {
    int val = blockIdx.x;
    int start_idx = scan[val];
    int end_idx = (val == k_alphabet_size - 1) ? n : scan[val + 1];
    for (int i = start_idx + threadIdx.x; i < end_idx; i += blockDim.x) {
        output[i] = (unsigned char)val;
    }
}

```

3. Параллельное заполнение - Каждый блок GPU обрабатывает один символ алфавита (256 блоков). Блок вычисляет свой диапазон в результирующем массиве на основе данных сканирования и последовательно записывает значения, используя объединенный доступ к памяти.

Результаты

Единицы измерения времени в сравнениях - ms.

1. Конфигурация <<< 1, 32 >>>

n	time (ms)
1000	0.183
1000000	43.566
10000000	369.425

2. Конфигурация <<< 256, 256 >>>

n	time (ms)
1000	0.066
1000000	0.472
10000000	6.939
100000000	34.227

3. Конфигурация <<< 512, 512 >>>

n	time (ms)
1000	0.073
1000000	0.552
10000000	0.747
100000000	4.745

4. Конфигурация <<< 1024, 1024 >>>

n	time (ms)
1000	0.084
1000000	0.653
10000000	0.844
100000000	4.969

```
!python3 -c "import struct, os; n=10_000_000; open('in.bin','wb').write(struct.pack('<I', n) + os.urandom(n))"
```

```
# 3anyck c nvprof
!nvprof ./sort_program < in.bin > out.bin
```

```
... ==11658== NVPROF is profiling process 11658, command: ./sort_program
GPU Kernel Execution Time: 0.844 ms
==11658== Profiling application: ./sort_program
==11658== Profiling result:
```

Type	Time(%)	Time	Calls	Avg	Min	Max	Name
GPU activities:	68.67%	5.8689ms	1	5.8689ms	5.8689ms	5.8689ms	[CUDA memcpy DtoH]
	23.88%	2.0407ms	1	2.0407ms	2.0407ms	2.0407ms	[CUDA memcpy HtoD]
	6.14%	524.74us	1	524.74us	524.74us	524.74us	ComputeHistogramKernel(unsigned char const *, int*, int)
	1.14%	97.376us	1	97.376us	97.376us	97.376us	FillOutputKernel(unsigned char*, int const *, int)
	0.15%	13.023us	1	13.023us	13.023us	13.023us	ComputeScanKernel(int const *, int*)
	0.02%	1.3440us	1	1.3440us	1.3440us	1.3440us	[CUDA memset]
API calls:	89.72%	99.684ms	4	24.921ms	3.4810us	99.510ms	cudaMalloc
	8.27%	9.1881ms	2	4.5941ms	2.2407ms	6.9474ms	cudaMemcpy
	0.98%	1.0882ms	4	272.04us	6.1350us	445.15us	cudaFree
	0.56%	621.35us	1	621.35us	621.35us	621.35us	cudaEventSynchronize
	0.21%	232.74us	3	77.579us	8.1510us	215.89us	cudaLaunchKernel
	0.18%	198.33us	114	1.7390us	118ns	82.744us	cuDeviceGetAttribute
	0.03%	28.294us	1	28.294us	28.294us	28.294us	cuDeviceGetName
	0.02%	18.625us	1	18.625us	18.625us	18.625us	cudaMemset
	0.01%	16.120us	2	8.0600us	645ns	15.475us	cudaEventCreate
	0.01%	11.373us	2	5.6860us	3.8390us	7.5340us	cudaEventRecord
	0.01%	5.9570us	1	5.9570us	5.9570us	5.9570us	cuDeviceGetPCIBusId
	0.00%	3.9460us	2	1.9730us	466ns	3.4800us	cudaEventDestroy
	0.00%	2.7570us	3	919ns	154ns	2.0920us	cudaEventGetCount
	0.00%	2.3100us	1	2.3100us	2.3100us	2.3100us	cudaEventElapsedTime
	0.00%	1.5400us	2	770ns	250ns	1.2900us	cuDeviceGet
	0.00%	739ns	1	739ns	739ns	739ns	cuDeviceTotalMem
	0.00%	432ns	1	432ns	432ns	432ns	cuModuleGetLoadingMode
	0.00%	351ns	1	351ns	351ns	351ns	cuDeviceGetUuid

Рис. 1 — Результат работы nvprof

```
#include <iostream>
#include <vector>
#include <chrono>
#include <csdio>

void RunCpuCountingSort(int n, unsigned char* input, unsigned char* output) {
    int histogram[256] = {0};

    auto start = std::chrono::high_resolution_clock::now();

    for (int i = 0; i < n; ++i) {
        histogram[input[i]]++;
    }

    int current_pos = 0;
    for (int val = 0; val < 256; ++val) {
        for (int i = 0; i < histogram[val]; ++i) {
            output[current_pos++] = (unsigned char)val;
        }
    }

    auto end = std::chrono::high_resolution_clock::now();
    std::chrono::duration<float, std::milli> duration = end - start;

    fprintf(stderr, "CPU Execution Time: %.3f ms\n", duration.count());
}

int main() {
    int n = 0;
    if (fread(&n, sizeof(int), 1, stdin) < 1) return 0;
    if (n <= 0) return 0;

    unsigned char* input = (unsigned char*)malloc(n);
    unsigned char* output = (unsigned char*)malloc(n);

    if (fread(input, 1, n, stdin) < (size_t)n) {
        free(input); free(output);
        return 1;
    }

    RunCpuCountingSort(n, input, output);

    fwrite(output, 1, n, stdout);

    free(input);
    free(output);
    return 0;
}
```

Writing sort_cpu.cpp

Рис. 2 — Сравнение с CPU без использования CUDA

Для теста с $n = 100\,000\,000$ и конфигурацией $\langle\langle\langle 1024, 1024\rangle\rangle\rangle$:

```
# 2. Подготовка большого тестового файла (100 млн элементов)
n = 100_000_000
print(f"Генерация теста для n = {n}...")
with open('big_test.bin', 'wb') as f:
    f.write(struct.pack('<I', n))
    f.write(os.urandom(n))

# 3. Запуск теста на CPU
print("\n--- ТЕСТ CPU ---")
!./sort_cpu < big_test.bin > /dev/null

# 4. Запуск теста на GPU
print("\n--- ТЕСТ GPU ---")
!./sort_gpu < big_test.bin > /dev/null

... Генерация теста для n = 100000000...

--- ТЕСТ CPU ---
CPU Execution Time: 128.809 ms

--- ТЕСТ GPU ---
GPU Kernel Execution Time: 4.969 ms
```

Рис. 3 — Сравнение с CPU по времени исполнения

Выводы

Основная область применения — задачи, где диапазон возможных значений данных значительно меньше количества самих элементов: группировка данных (Group By) и предварительная сортировка ключей в СУБД, построение гистограмм яркости/цвета для коррекции контрастности или сегментации, для сжатия данных.

Основные трудности при реализации:

- 1) При реализации Blelloch Scan в разделяемой памяти возникла сложность с обеспечением параллельного доступа. Трудности преодолены декомпозицией алгоритма на последовательные фазы запуска ядер
- 2) Проблема с производительностью при использовании атомарных операций была решена переносом промежуточных вычислений в разделяемую память.

На максимальном объеме входных элементов (около 512 МБ), GPU становится безальтернативным решением, так как позволяет обработать данные в реальном времени, в то время как CPU замедляется.

Конфигурации с числом блоков от 256 и до 1024 практически не отличаются по показателям времени.